Trabajo práctico #3: Seguridad, Monitoreo y Resolución de Problemas
Laboratorio de Investigación Gugler
Facultad de Ciencia y Tecnología - UADER
Alumno: Romero Gonzalo

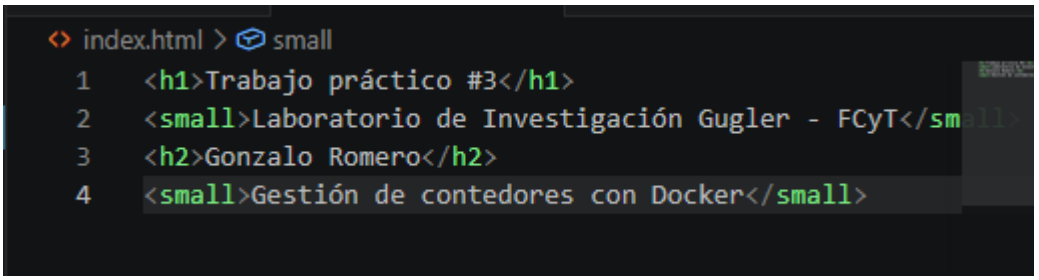
TP3: Seguridad, Monitoreo y Resolución de Problemas

En este trabajo práctico buscaremos afianzar los conocimientos adquiridos sobre los conceptos de Seguridad, Monitoreo y Resolución de Problemas.
Objetivos
Lograr familiarizarse con el entorno y las capacidades fundamentales de Docker respecto de:
• Mejores prácticas de seguridad
Gestión de secretos y variables de entorno
Escaneo de imágenes en busca de vulnerabilidades
Visualización de logs de contenedores
Errores comunes y soluciones
• Consejos para la depuración de contenedores
Desarrollo
A partir de tener la instalación completa de Docker en el equipo demuestre como realizar los puntos propuestos.
Como evidencia, realice capturas de pantalla de la terminal que uso para resolver la entrega.

Entrega
La entrega deberá ser en un documento de texto en formato PDF; en el documento deberá escribir el código utilizado y realizar capturas de pantalla de las actividades realizadas y donde se pueda visualizar como se llevó a cabo.

Ejercicio 1: Crear una imagen Docker
Crear una imagen Docker que ejecute una aplicación web que muestre un mensaje al acceder vía el puerto 8081 de su navegador web (use http://127.0.0.1:8081). Genere un archivo HTML como hicimos en el TP#1. Genere un Dockerfile para esta imagen. Utilice las instrucciones: ENV EXPOSE LABEL USER Y VOLUME

html:



Dockerfile:

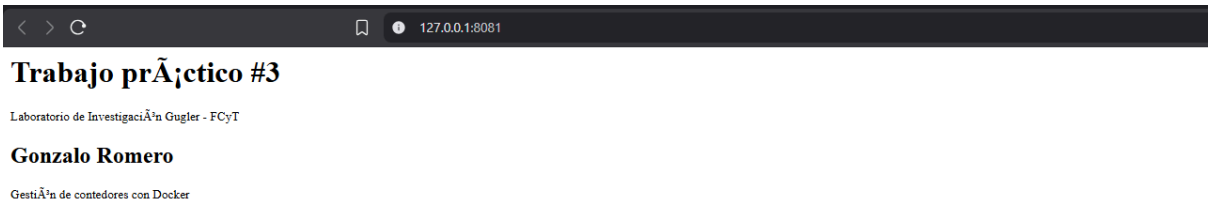
```
Dockerfile > ...
1   FROM nginx:latest
2   LABEL autor="Gonzalo Romero"
3   LABEL descripcion="Imagen para el TP3"
4   ENV AMBIENTE=laboratorio
5   COPY ./index.html /usr/share/nginx/html/index.html
6   VOLUME /usr/share/nginx/html
7   EXPOSE 80
8   USER root
```

comandos en terminal:

```
C:\Users\Gonza\Desktop\tps\DOCKER\tp3>docker build -t indextp3 .
[+] Building 1.1s (7/7) FINISHED                                                docker:desktop-linux
 => [internal] load build definition from Dockerfile                                           0.0s
 => => transferring dockerfile: 256B                                                           0.0s
 => [internal] load metadata for docker.io/library/nginx:latest                                0.1s
 => [internal] load .dockerignore                                                              0.0s
 => => transferring context: 2B                                                                0.0s
 => [1/2] FROM docker.io/library/nginx:latest@sha256:5aca99593157f4ae539a5dec1092a0ad8762f8e2eb1789085a13a0f56223  0.1s
 => => resolve docker.io/library/nginx:latest@sha256:5aca99593157f4ae539a5dec1092a0ad8762f8e2eb1789085a13a0f56223  0.1s
 => [internal] load build context                                                              0.1s
 => => transferring context: 32B                                                               0.0s
 => CACHED [2/2] COPY ./index.html /usr/share/nginx/html/index.html                            0.0s
 => exporting to image                                                                         0.4s
 => => exporting layers                                                                        0.0s
 => => exporting manifest sha256:3bfa7c9c9e9d26dc8dd7b9cc5d6cd365e32f9fcffedfeae5e906b6dfcce6c1cd  0.1s
 => => exporting config sha256:75f0bdf338ead0c1f6d978d8c9c3578e0e703e87a838d14a2ab8e731f80a05b9  0.1s
 => => exporting attestation manifest sha256:235d8daf20bf3387c9f36e03fd9deb6290de8fd4af87936ac725b78a3fe8b989  0.1s
 => => exporting manifest list sha256:fc5ab0240da16b7d571d074f1ac862e7c3c6c02d484300fae398a2426d95284b  0.1s
 => => naming to docker.io/library/indextp3:latest                                             0.0s
 => => unpacking to docker.io/library/indextp3:latest                                          0.0s
```

```
C:\Users\Gonza\Desktop\tps\DOCKER\tp3>docker run -d -p 8081:80 --name webtp3 indextp3
7d864808059ba5bbe21128091b21e5423d56e7109a7dfc7525739467c8b43e6e
```

abriendo http://127.0.0.1:8081/



Ejercicio 2: Mejores prácticas de seguridad
Aplica diversas prácticas para mejorar la seguridad en la imagen que creaste en el ejercicio anterior.

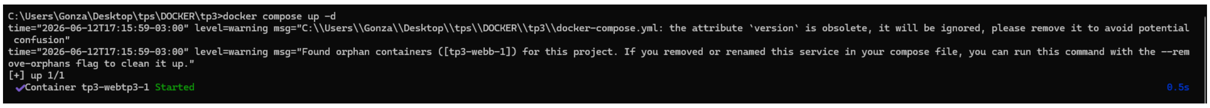
```
1   FROM nginx:latest
2   LABEL autor="Gonzalo Romero"
3   LABEL descripcion="Imagen para el TP3"
4   ENV AMBIENTE=laboratorio
5   COPY ./index.html /usr/share/nginx/html/index.html
6   VOLUME /usr/share/nginx/html
7   EXPOSE 80
8   USER root
```

El Dockerfile queda igual, porque se recomienda usar imagen oficial de dockerhub y, aparte, la más actualizada, y eso ya lo estoy haciendo.

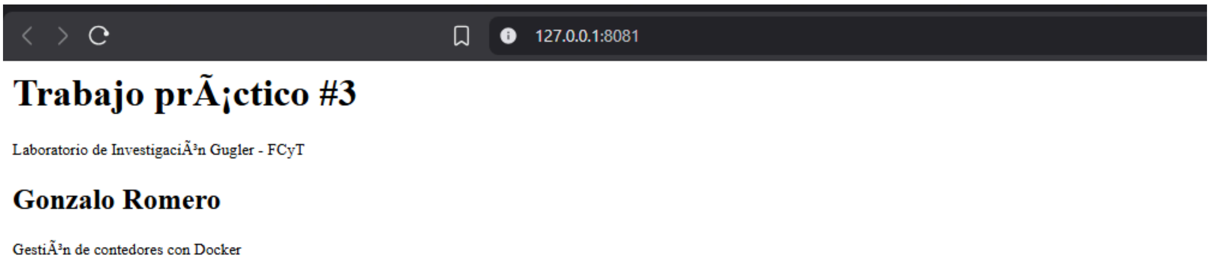
```
1   FROM nginx:latest
2   LABEL autor="Gonzalo Romero"
3   LABEL descripcion="Imagen para el TP3"
4   ENV AMBIENTE=laboratorio
5   COPY ./index.html /usr/share/nginx/html/index.html
6   VOLUME /usr/share/nginx/html
7   EXPOSE 80
8   RUN addgroup -S myuser && adduser -S myuser -G myuser
9   USER myuser
```

Se cambió el USER y se añadió el usuario myuser para poder NO usar root, porque se recomienda no usar contenedores desde root.
Hice un docker compose para limitar los recursos del contenedor.

se ejecuta en la terminal:



se ve así en la web:



añadí una network al docker compose:

Dockerfile U docker-compose.yml U

docker-compose.yml

```yaml
version: "3.8"
Run All Services
services:
  Run Service
  webtp3:
    image: indextp3
    ports:
      - "8081:80"
    networks:
      - my_network
    deploy:
      resources:
        limits:
          cpus: "0.5"
          memory: 512M

networks:
  my_network:
    driver: bridge
```

en la terminal:

```
C:\Users\Gonza\Desktop\tps\DOCKER\tp3>docker compose up -d
time="2026-06-12T17:19:06-03:00" level=warning msg="C:\\Users\\Gonza\\Desktop\\tps\\DOCKER\\tp3\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential
 confusion"
[+] up 0/1
 - Network tp3_my_network Creating                                                                                                                                                                    0.1s
[+] up 2/2-06-12T17:19:06-03:00" level=warning msg="Found orphan containers ([tp3-webb-1]) for this project. If you removed or renamed this service in your compose file, you can run this command with the --rem
 ✓Network tp3_my_network Created                                                                                                                                                                      0.1s
 ✓Container tp3-webtp3-1 Started                                                                                                                                                                      0.6s
```

en la web:

añadí secrets en el docker compose, por si surge la necesidad de usar contraseñas.

```yaml
version: "3.8"
Run All Services
services:
  Run Service
  webtp3:
    image: indextp3
    ports:
      - "8081:80"
    networks:
      - my_network
    deploy:
      resources:
        limits:
          cpus: "0.5"
          memory: 512M
    secrets:
      - cosas_secretas

networks:
  my_network:
    driver: bridge
secrets:
  cosas_secretas:
    file: ./cosas_secretas/contraseña.txt
```

corrí trivy por primera vez y me dió:

```
Report Summary
```

Target	Type	Vulnerabilities	Secrets
indextp3 (debian 13.5)	debian	284	-

```
Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)
```

para tratar de solucionar, cambié el dockerfile:

> Users > Gonza > Desktop > tps > DOCKER > tp3 > Dockerfile > ...

```
1   FROM nginx:alpine
2   LABEL autor="Gonzalo Romero"
3   LABEL descripcion="Imagen para el TP3"
4   ENV AMBIENTE=laboratorio
5   COPY ./index.html /usr/share/nginx/html/index.html
6   VOLUME /usr/share/nginx/html
7   EXPOSE 80
8   RUN addgroup -S myuser && adduser -S myuser -G myuser
9   USER myuser
```

al revisar de nuevo:

Target	Type	Vulnerabilities	Secrets
indextp3 (alpine 3.23.4)	alpine	31	-

```
Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)
```

bajamos las vulnerabilidades a 31 (las vulnerabilidades que quedan son, en su mayoria de riesgo “LOW” y, revisandolas, se solucionarán cuando actualicen la imagen de nginx:alpine, porque el alpine 3.5.6-r0 es la versión que se instala, y las vulnerabilidades que marca, en su mayoría, se solucionan en la 3.5.7-r0).

Ejercicio 3: Gestión de variables de entorno
Genera un archivo compose donde utilices una base de datos (la que prefiera) y configura las credenciales de acceso mediante un archivo .env
Cree una aplicación Flask que se conecte a la base de datos y liste alguna información creada por usted (también use las credenciales desde el archivo .env).

.env:

.env

```
# base de datos
DB_USER=mi_usuario_flask
DB_PASSWORD=clave_super_segura_123
DB_NAME=db_inventario
DB_HOST=db
DB_PORT=5432

# flask
FLASK_APP=app.py
FLASK_ENV=development
```

app.py:

app.py > ...

```python
import os
import time
import psycopg2
from flask import Flask, jsonify

app = Flask(__name__)

def get_db_connection():
    # Bucle de reintentos en caso de que PostgreSQL tarde unos segundos en iniciar
    retries = 5
    while retries > 0:
        try:
            conn = psycopg2.connect(
                host=os.environ.get('DB_HOST'),
                database=os.environ.get('DB_NAME'),
                user=os.environ.get('DB_USER'),
                password=os.environ.get('DB_PASSWORD'),
                port=os.environ.get('DB_PORT')
            )
            return conn
        except psycopg2.OperationalError:
            retries -= 1
            print("Esperando a la base de datos...")
            time.sleep(2)
    raise Exception("No se pudo establecer conexión con PostgreSQL")

def init_db():
    """Crea la tabla e inserta datos si está vacía."""
    conn = get_db_connection()
    cur = conn.cursor()

    cur.execute('''
        CREATE TABLE IF NOT EXISTS productos (
            id SERIAL PRIMARY KEY,
            nombre VARCHAR(100) NOT NULL,
            precio DECIMAL(10, 2) NOT NULL
        );
    ''')

    cur.execute('SELECT COUNT(*) FROM productos;')
    if cur.fetchone()[0] == 0:
        cur.execute("INSERT INTO productos (nombre, precio) VALUES ('Notebook Intel i7', 1450.00);")
        cur.execute("INSERT INTO productos (nombre, precio) VALUES ('Teclado Mecánico RGB', 85.50);")
        cur.execute("INSERT INTO productos (nombre, precio) VALUES ('Monitor Curvo 27\"', 320.00);")
        conn.commit()

    cur.close()
    conn.close()
```

app.py > ...

```python
def init_db():
    cur.close()
    conn.close()

@app.route('/')
def home():
    return jsonify({
        "status": "online",
        "message": "Navega a /productos para listar la información de la BD."
    })

@app.route('/productos', methods=['GET'])
def listar_productos():
    try:
        conn = get_db_connection()
        cur = conn.cursor()
        cur.execute('SELECT id, nombre, precio FROM productos;')
        rows = cur.fetchall()

        # Mapeo de la tupla a formato JSON nativo
        lista_productos = []
        for row in rows:
            lista_productos.append({
                "id": row[0],
                "nombre": row[1],
                "precio": float(row[2])
            })

        cur.close()
        conn.close()
        return jsonify(lista_productos)
    except Exception as e:
        return jsonify({"error": str(e)}), 500

if __name__ == '__main__':
    init_db()
    app.run(host='0.0.0.0', port=5000)
```

docker compose:

docker-compose.yml

```yaml
version: '3.8'

services:
  db:
    image: postgres:15-alpine
    container_name: postgres_db
    restart: always
    environment:
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: ${DB_NAME}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"

  web:
    build: .
    container_name: flask_app
    restart: always
    ports:
      - "5000:5000"
    environment:
      - DB_USER=${DB_USER}
      - DB_PASSWORD=${DB_PASSWORD}
      - DB_NAME=${DB_NAME}
      - DB_HOST=${DB_HOST}
      - DB_PORT=${DB_PORT}
    depends_on:
      - db

volumes:
  postgres_data:
```

Dockerfile:

Dockerfile > ...

```dockerfile
FROM python:3.10-slim

WORKDIR /app

# Instalar dependencias necesarias para compilar/conectar bases de datos
RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc \
    libpq-dev \
    && rm -rf /var/lib/apt/lists/*

# Instalar los paquetes de Python
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copiar el código de nuestra app
COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
```

requirements.txt:

requirements.txt

```
Flask==3.0.2
psycopg2-binary==2.9.9
```

creación de la imagen:

```
C:\Users\Gonza\Desktop\tps\DOCKER\tp3\ejercicio 4>docker build -t ejercicio4tp4 .
[+] Building 71.0s (12/12) FINISHED                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                  0.1s
 => => transferring dockerfile: 494B                                                                  0.0s
 => [internal] load metadata for docker.io/library/python:3.10-slim                                   2.2s
 => [auth] library/python:pull token for registry-1.docker.io                                         0.0s
 => [internal] load .dockerignore                                                                     0.1s
 => => transferring context: 2B                                                                       0.0s
 => [1/6] FROM docker.io/library/python:3.10-slim@sha256:09304d54d98baaa86d14fce52a168ee32b712e20e4e  5.0s
 => => resolve docker.io/library/python:3.10-slim@sha256:09304d54d98baaa86d14fce52a168ee32b712e20e4e  0.1s
 => => sha256:761b88a3de45a6f5174e0c4d1df0d0e914f455f182840c27d50ddc4d45fb88a4 249B / 249B            0.2s
 => => sha256:3c58578f39599387ef9d2d5ba89b53cfcc4e1015508e73dc63d008b672d95bc9 13.82MB / 13.82MB      1.1s
 => => sha256:f41c0231c824bc5b8565006225367cff6d5d78dd84a6292a459566a78f4ad0cc 1.29MB / 1.29MB        0.8s
 => => sha256:72c03230f1363a3fb61d2f98504cf168bad3fe22f511ad2005dc021515d7ce97 29.79MB / 29.79MB      2.2s
 => => extracting sha256:72c03230f1363a3fb61d2f98504cf168bad3fe22f511ad2005dc021515d7ce97              1.1s
 => => extracting sha256:f41c0231c824bc5b8565006225367cff6d5d78dd84a6292a459566a78f4ad0cc              0.2s
 => => extracting sha256:3c58578f39599387ef9d2d5ba89b53cfcc4e1015508e73dc63d008b672d95bc9              0.8s
 => => extracting sha256:761b88a3de45a6f5174e0c4d1df0d0e914f455f182840c27d50ddc4d45fb88a4             0.1s
 => [internal] load build context                                                                     0.2s
 => => transferring context: 2.81kB                                                                   0.0s
 => [2/6] WORKDIR /app                                                                                5.7s
 => [3/6] RUN apt-get update && apt-get install -y --no-install-recommends     gcc     libpq-dev     && rm -rf /  37.5s
 => [4/6] COPY requirements.txt .                                                                     5.7s
 => [5/6] RUN pip install --no-cache-dir -r requirements.txt                                          3.9s
 => [6/6] COPY app.py .                                                                               0.2s
 => exporting to image                                                                                9.4s
 => => exporting layers                                                                               7.4s
 => => exporting manifest sha256:050a5e44d11e9145fa561aac4d27a4281019140634 9d5fdc2fecd728f207bd5c     0.1s
 => => exporting config sha256:e0737fcf2fb200d31c325c3a5c68242316dd2221dc3c5990999defb764703aad       0.1s
 => => exporting attestation manifest sha256:24b22effe395c88bc3e8363347bab76a06caea035be544c091d7e110d149c874  0.1s
 => => exporting manifest list sha256:a523d393fb1e9da2f9c8640e17508568869618bc3f151aa146f840caa8b58bfb  0.1s
 => => naming to docker.io/library/ejercicio4tp4:latest                                               0.0s
 => => unpacking to docker.io/library/ejercicio4tp4:latest                                            1.7s
```

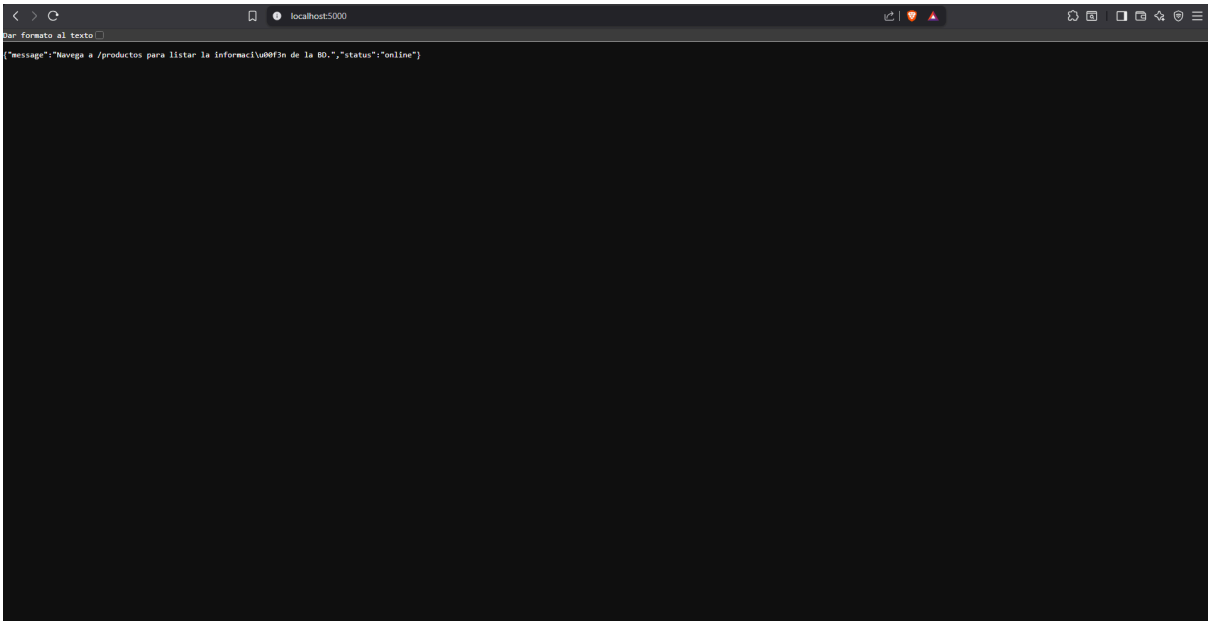
contenedor:

```
C:\Users\Gonza\Desktop\tps\DOCKER\tp3\ejercicio 4>docker compose up
time="2026-06-13T20:28:14-03:00" level=warning msg="C:\\Users\\Gonza\\Desktop\\tps\\DOCKER\\tp3\\ejercicio 4\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to av
oid potential confusion"
[+] up 13/13
 ✔Image postgres:15-alpine Pulled                                                                     17.4s
#1 [internal] load local bake definitions
#1 reading from stdin 559B done
#1 DONE 0.0s

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 494B done
#2 DONE 0.0s

#3 [internal] load metadata for docker.io/library/python:3.10-slim
#3 DONE 0.7s

#4 [internal] load .dockerignore
#4 transferring context: 2B done
#4 DONE 0.0s

#5 [internal] load build context
#5 transferring context: 63B done
#5 DONE 0.1s

#6 [1/6] FROM docker.io/library/python:3.10-slim@sha256:09304d54d98baaa86d14fce52a168ee32b712e20e4e82f7ec168799aa6060fd1
#6 resolve docker.io/library/python:3.10-slim@sha256:09304d54d98baaa86d14fce52a168ee32b712e20e4e82f7ec168799aa6060fd1
#6 resolve docker.io/library/python:3.10-slim@sha256:09304d54d98baaa86d14fce52a168ee32b712e20e4e82f7ec168799aa6060fd1 0.1s done
#6 DONE 0.1s

#7 [2/6] WORKDIR /app
#7 CACHED

#8 [3/6] RUN apt-get update && apt-get install -y --no-install-recommends     gcc     libpq-dev     && rm -rf /var/lib/apt/lists/*
#8 CACHED

#9 [4/6] COPY requirements.txt .
#9 CACHED

#10 [5/6] RUN pip install --no-cache-dir -r requirements.txt
#10 CACHED

#11 [6/6] COPY app.py .
#11 CACHED

#12 exporting to image
#12 exporting layers done
#12 exporting manifest sha256:c027e26fd0a90775661342e1c0dc524a7c395dbea62df5e683487d00a5b25a07 0.1s done
#12 exporting config sha256:65adad87ac70017c4a45ccb925ae0a35a4b36050548b5bed56e2bc5627a55579
#12 exporting config sha256:65adad87ac70017c4a45ccb925ae0a35a4b36050548b5bed56e2bc5627a55579 0.1s done
#12 exporting attestation manifest sha256:7a752c7315f88014f8f0374140d66c46f599a3f9ec4e595d2094b0905f3c99f4
```

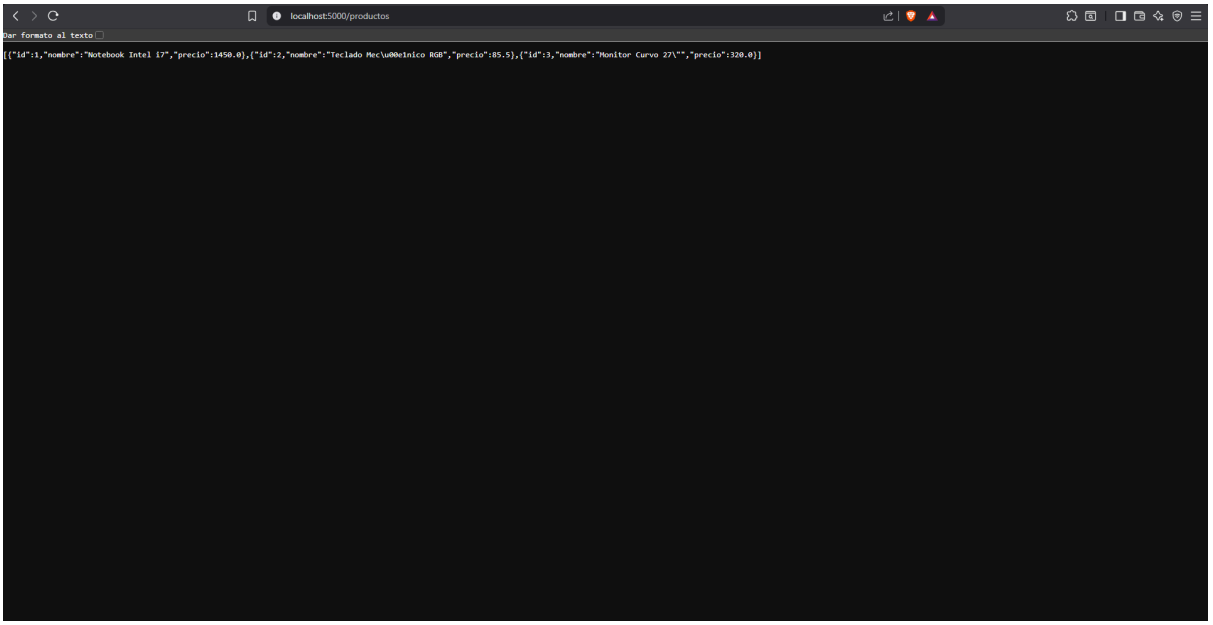
```
#12 naming to docker.io/library/ejercicio4-web:latest done
#12 unpacking to docker.io/library/ejercicio4-web:latest 0.0s done
#12 DONE 0.4s

[+] up 18/18g provenance for metadata file
 ✔Image postgres:15-alpine        Pulled                                                              17.4s
 ✔Image ejercicio4-web            Built                                                                2.4s
 ✔Network ejercicio4_default      Created                                                              0.1s
 ✔Volume ejercicio4_postgres_data Created                                                              0.0s
 ✔Container postgres_db           Created                                                             20.3s
 ✔Container flask_app             Created                                                              0.2s
Attaching to flask_app, postgres_db
postgres_db  | The files belonging to this database system will be owned by user "postgres".
postgres_db  | This user must also own the server process.
postgres_db  |
postgres_db  | The database cluster will be initialized with locale "en_US.utf8".
postgres_db  | The default database encoding has accordingly been set to "UTF8".
postgres_db  | The default text search configuration will be set to "english".
postgres_db  |
postgres_db  | Data page checksums are disabled.
postgres_db  |
postgres_db  | fixing permissions on existing directory /var/lib/postgresql/data ... ok
postgres_db  | creating subdirectories ... ok
postgres_db  | selecting dynamic shared memory implementation ... posix
postgres_db  | selecting default max_connections ... 100
postgres_db  | selecting default shared_buffers ... 128MB
postgres_db  | selecting default time zone ... UTC
postgres_db  | creating configuration files ... ok
postgres_db  | running bootstrap script ... ok
postgres_db  | sh: locale: not found
postgres_db  | 2026-06-13 23:28:56.357 UTC [35] WARNING:  no usable system locales were found
postgres_db  | performing post-bootstrap initialization ... ok
postgres_db  | syncing data to disk ... ok
postgres_db  |
postgres_db  |
postgres_db  | Success. You can now start the database server using:
postgres_db  |
postgres_db  |     pg_ctl -D /var/lib/postgresql/data -l logfile start
postgres_db  |
postgres_db  | initdb: warning: enabling "trust" authentication for local connections
postgres_db  | initdb: hint: You can change this by editing pg_hba.conf or using the option -A, or --auth-local and --auth-host, the next time you run initdb.
postgres_db  | waiting for server to start....2026-06-13 23:28:58.834 UTC [41] LOG:  starting PostgreSQL 15.18 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit
postgres_db  | 2026-06-13 23:28:58.842 UTC [41] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
postgres_db  | 2026-06-13 23:28:58.872 UTC [44] LOG:  database system was shut down at 2026-06-13 23:28:57 UTC
postgres_db  | 2026-06-13 23:28:58.887 UTC [41] LOG:  database system is ready to accept connections
postgres_db  |  done
postgres_db  | server started
postgres_db  | CREATE DATABASE
postgres_db  |
postgres_db  |
postgres_db  | /usr/local/bin/docker-entrypoint.sh: ignoring /docker-entrypoint-initdb.d/*
```

```
postgres_db  | /usr/local/bin/docker-entrypoint.sh: ignoring /docker-entrypoint-initdb.d/*
postgres_db  |
postgres_db  | waiting for server to shut down...2026-06-13 23:28:59.077 UTC [41] LOG:  received fast shutdown request
postgres_db  | .2026-06-13 23:28:59.088 UTC [41] LOG:  aborting any active transactions
postgres_db  | 2026-06-13 23:28:59.090 UTC [41] LOG:  background worker "logical replication launcher" (PID 47) exited with exit code 1
postgres_db  | 2026-06-13 23:28:59.091 UTC [42] LOG:  shutting down
postgres_db  | 2026-06-13 23:28:59.101 UTC [42] LOG:  checkpoint starting: shutdown immediate
postgres_db  | 2026-06-13 23:28:59.609 UTC [42] LOG:  checkpoint complete: wrote 921 buffers (5.6%); 0 WAL file(s) added, 0 removed, 0 recycled; write=0.043 s, sync=0.435 s, total=0.519 s; sync files=301, long
est=0.117 s, average=0.002 s; distance=4239 kB, estimate=4239 kB
postgres_db  | 2026-06-13 23:28:59.623 UTC [41] LOG:  database system is shut down
postgres_db  |  done
postgres_db  | server stopped
postgres_db  |
postgres_db  | PostgreSQL init process complete; ready for start up.
postgres_db  |
postgres_db  | 2026-06-13 23:28:59.736 UTC [1] LOG:  starting PostgreSQL 15.18 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit
postgres_db  | 2026-06-13 23:28:59.736 UTC [1] LOG:  listening on IPv4 address "0.0.0.0", port 5432
postgres_db  | 2026-06-13 23:28:59.736 UTC [1] LOG:  listening on IPv6 address "::", port 5432
postgres_db  | 2026-06-13 23:28:59.753 UTC [1] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
postgres_db  | 2026-06-13 23:28:59.771 UTC [57] LOG:  database system was shut down at 2026-06-13 23:28:59 UTC
postgres_db  | 2026-06-13 23:28:59.790 UTC [1] LOG:  database system is ready to accept connections
flask_app    | Esperando a la base de datos...
flask_app    | Esperando a la base de datos...
flask_app    |  * Serving Flask app 'app'
flask_app    |  * Debug mode: off
flask_app    | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
flask_app    |  * Running on all addresses (0.0.0.0)
flask_app    |  * Running on http://127.0.0.1:5000
flask_app    |  * Running on http://172.22.0.3:5000
flask_app    | Press CTRL+C to quit
postgres_db  | 2026-06-13 23:33:59.820 UTC [55] LOG:  checkpoint starting: time
postgres_db  | 2026-06-13 23:34:08.793 UTC [55] LOG:  checkpoint complete: wrote 91 buffers (0.6%); 0 WAL file(s) added, 0 removed, 0 recycled; write=8.849 s, sync=0.074 s, total=8.973 s; sync files=50, longes
t=0.012 s, average=0.002 s; distance=392 kB, estimate=392 kB
```

web:



web en /productos:

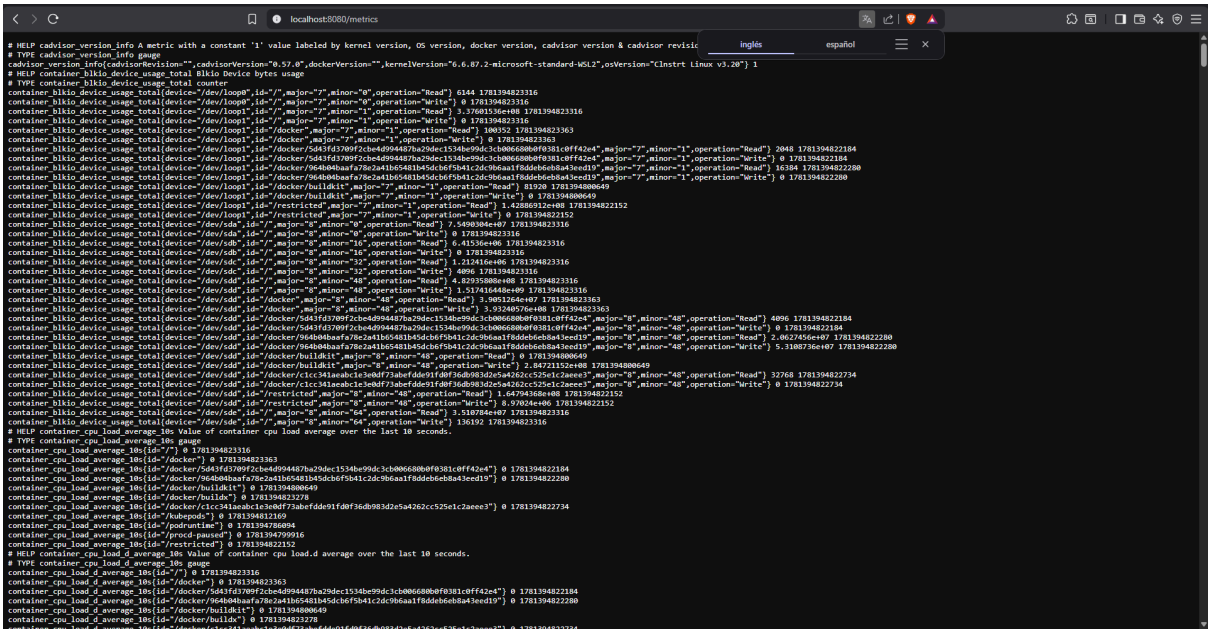


Ejercicio 4: Monitoreo de recursos de contenedores
Genera un contenedor con la imagen cAdvisor y muestra como puedes monitorizar tu instalación.
Añadí cAdvisor en el docker compose del ejercicio anterior:

docker-compose.yml

```yaml
services:
  web:
    container_name: flask_app
    restart: always
    ports:
      - "5000:5000"
    environment:
      - DB_USER=${DB_USER}
      - DB_PASSWORD=${DB_PASSWORD}
      - DB_NAME=${DB_NAME}
      - DB_HOST=${DB_HOST}
      - DB_PORT=${DB_PORT}
    depends_on:
      - db
  ▷Run Service
  cadvisor:
    image: cleanstart/cadvisor:latest
    container_name: cadvisor
    restart: always
    ports:
      - "8080:8080"
    volumes:
      - /:/rootfs:ro
      - /var/run:/var/run:ro
      - /sys:/sys:ro
      - /var/lib/docker/:/var/lib/docker:ro
      - /dev/disk/:/dev/disk:ro
    devices:
      - /dev/kmsg
    privileged: true

volumes:
  postgres_data:
```

http://localhost:8080/metrics, cadvisor arroja:



Ejercicio 5: Visualización de logs de contenedores
Investigue como y agregue logs dentro de la aplicación Flask del Ejercicio 3 para poder visualizarlos con el comando logs de Docker. Muestre los últimos logs de tus contenedores.

se añadió, en app.py:

```python
# 1. Configuración del sistema de Logging para Docker
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s [%(levelname)s] %(message)s',
    handlers=[
        logging.StreamHandler(sys.stdout) # Envía los logs directamente a la salida estándar
    ]
)
logger = logging.getLogger(__name__)

app = Flask(__name__)
```

se modificó el Dockerfile:

Dockerfile > ...

```
1   FROM python:3.10-slim
2
3   WORKDIR /app
4
5   ENV PYTHONUNBUFFERED=1
6
7   RUN apt-get update && apt-get install -y --no-install-recommends \
8       gcc \
9       libpq-dev \
10      && rm -rf /var/lib/apt/lists/*
11
12  COPY requirements.txt .
13  RUN pip install --no-cache-dir -r requirements.txt
14
15  COPY app.py .
16
17  EXPOSE 5000
18
19  CMD ["python", "app.py"]
```

al usar docker logs en la terminal, sale:

```
C:\Users\Gonza>docker logs flask_app
2026-06-14 00:04:22,122 [INFO] Intentando conectar a la base de datos PostgreSQL...
2026-06-14 00:04:22,178 [INFO] ¡Conexión establecida con éxito con PostgreSQL!
2026-06-14 00:04:22,178 [INFO] Verificando existencia de la tabla 'productos'...
2026-06-14 00:04:22,214 [INFO] La tabla ya contiene 3 registros. No se requiere inicialización.
2026-06-14 00:04:22,214 [INFO] Iniciando servidor web Flask en el puerto 5000...
 * Serving Flask app 'app'
 * Debug mode: off
2026-06-14 00:04:22,220 [INFO] WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.22.0.4:5000
2026-06-14 00:04:22,220 [INFO] Press CTRL+C to quit

C:\Users\Gonza>
```